

Extensions to Class Template Argument Deduction

This paper proposes several recommended extensions to [Class Template Argument Deduction](#)

Deduction guides for function templates

We propose allowing deduction guides for function templates. In addition to the (not to be underestimated) value of increased consistency, we provide the following motivating use cases.

Example: Unwrapping reference wrappers

`std::reference_wrapper` is commonly used to effect pass-by-reference to function templates with generic parameter types. Writing a function template that can be used this way is naturally simplified with deduction guides.

```
template <typename T> void f(T t) { t.foo(); /* ... */ }
template <T> f(reference_wrapper<T>) -> f<T &>;
```

Now `f` will unwrap reference wrappers arguments.

```
struct X { void foo(); /* ... */ };
int main()
{
    X x;
    f(ref(x)); // Passes by reference as desired. Ill-formed without deduction guide
}
```

Note: In C++ 17, something similar may be accomplished by rewriting `f` as

```
// Helper for converting reference wrappers to references
template <typename T> struct unwrap { using type = T };
template <typename T> struct unwrap<std::reference_wrapper<T>> { using type = T & };
template <typename T> using unwrap_t = typename unwrap<T>::type;

template <typename T> void f(T t)
{
    remove_reference_t<unwrap_t<T>> &actualT{t};
    actualT.foo(); /* ... */
}
```

However, not only is this more cumbersome in our opinion, it requires that the user of `f` look inside the implementation of `f` to understand that it conforms to the “reference_wrapper mini-language”, whereas the proposed version advertises that in the interface.

Example: Efficient parameter passing

For function templates that accept a `const` argument, it is generally most efficient to pass small argument types, such as `int`, by value and large or uncopyable types by `const &`. The Boost Call Traits library has a `call_traits<T>::param_type` traits that gives the most efficient way to pass, but it is awkward to use.

```
// C++17
template <typename T> void do_f(T t); // Implements f

template <typename T> inline void f(T && t)
{
    do_f<typename boost::call_traits<decay_t<T>>::type>(t);
}
```

With deduction guides, this again naturally simplifies to a form that no longer requires examining the body of `f`

```
template <typename T> void f(T t); // Implement here. No need to delegate
template <typename T> f(T t) -> f<typename boost::call_traits<T>::type>;
```

Example: Delayed forwarding

The following example, due to Richard Smith, applies function deduction guides to delayed forwarding in order to convert to the callee's types in the caller. E.g., to eliminate the need to use `std::ref` if the callee expects a reference (this has been known to [bite](#) users of `async` as well as of `thread`'s constructor template).

```
// Assume p0945r0 and std::experimental::invocation_type
template<typename F, typename R, typename ...T>
future<R> call_later(F fn, T ...t);

template<typename F, typename T> struct call_later_helper;
template<typename F, typename R, typename ...T> struct call_later_helper<R(T...)> {
    using call_later = ::call_later<F, R, T...>;
};

template<typename F, typename ...T>
call_later(F fn, T &&...t) -> call_later_helper<F, invocation_type_t<F, T&&...>>::call_later;
```

Argument deduction for function templates

For example, we believe the following code would be natural and useful

```
template <typename T> void f(optional<T>);
template <typename CharT, typename Traits> void g(basic_string_view<CharT, Traits>);
/* ... */
f(7); // Would like to deduce f<int> with arg of optional<int>
g("foo"); // Would like to deduce g<char, char_traits<char>> with arg of string_view
```

Note that pending a consensus around terse notation for concepts, this paper does not propose allowing declarations of function templates without the template keyword.

```
void f(optional); // Not part of this proposal/* ... */
f(7); // Does not deduce f<int>
```

Return type deduction for functions

We propose extending C++ return type deduction to allow Class Template Argument Deduction as code like the following appears both useful and natural to us.

```
tuple f() { return {3, 5}; } // tuple<int, int>
```

All return statements that deduce should deduce the same type, and at least one return statement should be able to deduce a return type.

```
optional f() { return 7; } // optional<int>
auto g() -> optional { if(noValue()) return {} else return 7; } // optional<int>
optional h() { if(foo()) return "bar" else return 7; } // Ill-formed: Ambiguous
```

Class Template Argument Deduction for aggregates

It seems awkward to us that the following does not work

```
template<typename T, typename U> struct TU { T t; U u; };
TU<int, double> tu{1, 2.3}; // OK
TU tu2{1, 2.3}; // Ill-formed
```

especially when the following example does

```
template<typename T, typename U> struct TU_ { TU_{T t, U u} : t{t}, u{u} {}; T t; U u; };
TU_<int, double> tu{1, 2.3}; // OK
TU_ tu2{1, 2.3}; // OK. TU_<int, double>
```

We propose that deduction takes place from the arguments. Rather than a deduction guide *per se*, this will need to use the same magic that aggregate initializers use to support the particular notations for aggregate initialization like [designated initializers](#).

```
template<typename T, typename U> struct TUT { T t1; U u; T t2 = T{} };
TUT tut1 = { 1, 2.3, 2 }; // TUT<int, double>{1, 2.3, 2}
TUT tut2 = {.t1 = 3, u = 2.3 }; // TUT<int, double>{3, 2.3, 0}
```

Suppressing deduction

Deleted deduction guides

When C++ automatically generates functions or methods, it generally allows the programmer to suppress the generation of function bodies through the use of `= delete`; . E.g.

```
template <typename T> struct A { A(T t); /* ... */ };

template <typename T>
auto makeA(T t) -> A<T> { return t; }
auto makeA(double) = delete;
auto makeA(int) -> A<int> = delete;
template <> makeA<long> -> A<long> = delete;
```

It seems very odd that ordinary template functions can be deleted but deduction guides cannot. Indeed, this was proposed in [Template argument deduction for class templates \(Rev. 7\)](#) with a vote in favor of adding to C++17 as a DR (we would also be OK with C++20 as a ship vehicle). For consistency, the same rules should apply as for deleting function templates.

We contend that supporting `= delete` for deduction guides not only increases consistency but has worthwhile use cases as well.

Example: `unique_ptr`

If we imagine a slightly simplified version of `unique_ptr` that had a `unique_ptr<T>::unique_ptr(T *)` constructor, this would yield a type-unsafe deduction of `T` as `int` when deducing from an `int *` even if `int[]` would be correct. As a result, library writers would have to resort to ugly workarounds such as deducing an invalid type like `unique_ptr<void>`. Allowing deduction guides to be deleted would provide a clear and uniform solution to such situations with either of the following:

```
template <typename T> unique_ptr(T *) = delete; // OK
template <typename T> unique_ptr(T *) -> unique_ptr<T> = delete; // Also OK
```

Suppressing participation in overload resolution

When a function or method is deleted with `= delete`, it still participates on overload resolution. However, it may be desirable to delete it from the overload set as well. There was a long debate in Kona over whether `reverse_iterator(reverse_iterator<T>)` should deduce to `reverse_iterator<T>` (copying) or `reverse_iterator<reverse_iterator<T>>` (wrapping). Although it was voted (we believe correctly) that copying should be preferred, it was generally accepted that there were use cases where wrapping would make sense. If we wanted to enshrine this in the type system with a `wrapping_reverse_iterator<T>`, we would want to entirely obliterate the implicitly-generated deduction guide from the copy constructor with a different type of deletion (bikeshed: `= default delete`) that also removed the guide from overload resolution.

```
template <typename T>
wrapping_reverse_iterator(wrapping_reverse_iterator<T> const &) -> wrapping_reverse_iterator<T> = default delete;
```

The deduction guide must exactly match the existing (implicitly-generated) deduction guide that it is obliterating, including explicitly specifying the return type. (This seems most consistent with the existing rules for deleting specialization of function templates).

We believe this would be valuable for general functions and function templates, not just deduction guides. For example, a hypothetical wrapping version of `any` would like to remove the copy constructor from overload resolution.

```
struct wrapping_any {
    wrapping_any(wrapping_any const &) = default delete;
    /* ... */
};

any a{3}; // contains an int
any b{a}; // also contains an int

wrapping_any wa{3}; // contains an int
wrapping_any wb{wa}; // contains a wrapping_any
```

Example: `valarray`

Consider the following example from “P0433R1: Toward a resolution of US7 and US14: Integrating template deduction for class templates into the standard library.”

```
int iarr[] = {1, 2, 3};
int *ip = iarr;
valarray va(ip, 3); // Deduces valarray<int>
```

The point is that the `valarray<T>::valarray(const T *, size_t)` constructor is a better match than the `valarray<T>::valarray(const T &, size_t)` constructor, so `valarray<int>` is preferred over `valarray<int *`. Given the typical usage of `valarray`, we believe that this is a reasonable default, and that is what we recommend in P0433R1.

However, it is worth noting that the proposed deletion mechanisms would have allowed us to make any other choice desired. For example, suppose we had wanted this deduction to be ambiguous for `valarray` or a similar class. The natural way to do this would be to delete the implicit deduction guide that takes a pointer.

```
template <T> valarray(const T *, size_t) -> valarray<T> = delete;
```

Likewise, supposed we had wished to deduce `valarray<int *`. We could accomplish this as follows:

```
template <T> valarray(const T *, size_t) -> valarray<T> = default delete;
```

Class Template Argument Deduction and partially-specialized template argument lists

In C++17, we deferred allowing CTAD to be applied in declarations with partially-specialized template argument lists out of concern that interactions with default arguments could create a breaking change. Consider the following example

```
template <typename T, typename U = T> struct A { A(T t, U u = U{}) {} };
template <typename T, typename U = T> A<T, U> makeA(T t, U u = U{}) { return A(t, u); }

auto a1 = A<int>(2, 3.5); // C++17: A<int, int>
auto a2 = makeA<int>(2, 3.5); // A<int, double>
```

To add support for partially-specialized template argument lists in declarations, we would have to decide what we want `A<int>(2, 3.5)` to deduce. The options are

1. `A<int, int>` as it currently does.
2. `A<int, double>` as a function template like `makeA()` does.
3. Make it ambiguous when options 1 and option 2 would give a different answer.

We can compare these approaches with some motivating examples (assuming the deduction guides like those in [P0433R3](#))

```
map<string, int> caseInsensitiveWordCounts(boost::algorithm::ilexical_compare{});
auto v = vector<int>(MyAlloc<int>{});
```

After considering examples such as the above, we propose adopting approach 2 for the following reasons.

- In both of the motivating examples, approach 2 deduces correctly while 1 and 3 fail to compile.
- We think approach 2 makes the language more consistent by giving deduction guides the same deduction rules as function templates while options 1 and 3 (as well as current C++17 behavior) create differences in how deduction guides and make functions handle partial specializations.
- In cases where both approach 1 and 2 compile, we think that any differences in legacy code have a high likelihood of being due to bugs. E.g., in the (admittedly artificial) `struct A` example, the user almost certainly expects `a2` to be `A<int, double>` else why would they be passing it 3.5?
- If the legacy C++17 behavior is desired for some class, that can still be achieved with a deduction guide. However, the opposite is not the case if deduction guides cannot be used in partially-specialized template argument lists in declarations.

Class Template Argument Deduction for alias templates

C++17 allows deduction for vector in cases such as

```
vector v = {1, 2, 3}; // vector<int>
```

However, equally desirable code for `pmr::vector` fails because `pmr::vector` is an alias template and not a class template

```
pmr::vector v = {1, 2, 3}; // Ill formed. pmr::vector is not a class template
```

Furthermore, there is no way to perform CTAD for `pmr::vector` within C++17 language rules because alias templates cannot use CTAD. We believe that examples such as this motivate supporting CTAD for alias templates. We do note as a bikeshed that Class Template Argument Deduction is a misnomer for alias templates.

Deducing from inherited constructors

Users may find it surprising that the following code is ill-formed.

```
template <typename T> struct A { A(T); };
template <typename T> struct B : public A<T> { using A<T>::A; };
A a{3}; // Ill-formed. Inherited constructors do not implicitly define deduction guides
```

This can make creating thin wrappers for classes (e.g., to just override a single method) cumbersome and error-prone, especially since the author of the derived class may need to manually replicate not only all the user-defined deduction guides of the base class but also all of the base class' implicitly-defined deduction guides as well. As a result, we propose that inheriting constructors from a base class also inherits their implicit and explicit deduction guides as well.

Of course, this only applies in cases where the derived classes template arguments are determined from the base class' template arguments. For example,

```
template<typename T> struct B { B(T t); /* ... */};
template<typename T, typename U> struct D : B<U> {
    using B<U>::B;
    T t{};
};

B b{7}; // B<int>
D d{7}; // Ill-formed
D<double> d2{7}; // D<double, int> by CTAD proposal for partially-specialized argument lists above
```

We suspect the feature will be useful and that counterexamples such as `struct D` will be the exception since inherited constructors also tend to require (admittedly with some differences) that the construction of the derived class be largely determined by the construction of the base

class. Indeed, the counterexample above was constructed through the at least somewhat esoteric technique of combining inherited constructors with non-static data member initializers for members of dependent types.